

Experiment 1: Installation of Reinforcement Learning Development Environment

Aim: To install and configure Anaconda Navigator, Python, TensorFlow, Keras, Gymnasium (OpenAI Gym), NumPy, Matplotlib, and other required libraries, and verify the environment by executing a simple Reinforcement Learning program

Program:

```
import tensorflow as tf

import gymnasium as gym

import numpy as np

import matplotlib.pyplot as plt

print("TensorFlow Version :", tf.__version__)

print("Gymnasium Imported Successfully")

print("NumPy Version :", np.__version__)

print("Matplotlib Imported Successfully")

env = gym.make("FrozenLake-v1")

observation, info = env.reset()

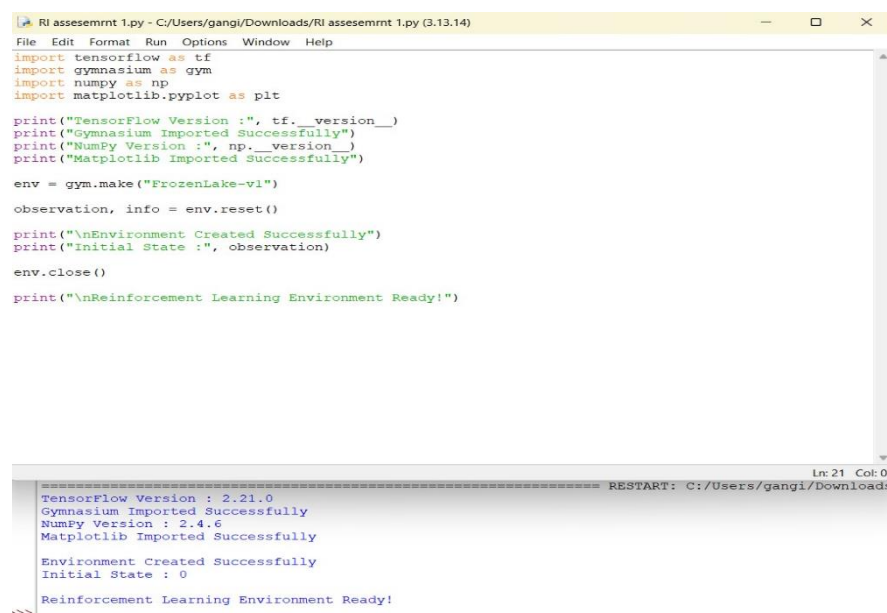
print("\nEnvironment Created Successfully")

print("Initial State :", observation)

env.close()

print("\nReinforcement Learning Environment Ready!")
```

Output:



The screenshot shows a code editor window titled "RI assesemrnt 1.py - C:/Users/gangli/Downloads/RI assesemrnt 1.py (3.13.14)". The code is the same as the one in the "Program" section. Below the code editor is a terminal window showing the output of the script. The output is as follows:

```
TensorFlow Version : 2.21.0
Gymnasium Imported Successfully
NumPy Version : 2.4.6
Matplotlib Imported Successfully

Environment Created Successfully
Initial State : 0

Reinforcement Learning Environment Ready!
->>
```

Experiment 2: Familiarization with OpenAI Gym/Gymnasium Environments

AIM:

To create, initialize, and interact with standard Reinforcement Learning environments such as FrozenLake, CartPole, and Mountain Car using Gymnasium to understand the concepts of states, actions, rewards, and episodes.

PROGRAM:

```
import gymnasium as gym

env = gym.make("FrozenLake-v1", render_mode="ansi")

state, info = env.reset()

print("Initial State:", state)

for step in range(10):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print("-----")

    print("Step:", step + 1)

    print("Action:", action)

    print("Next State:", next_state)

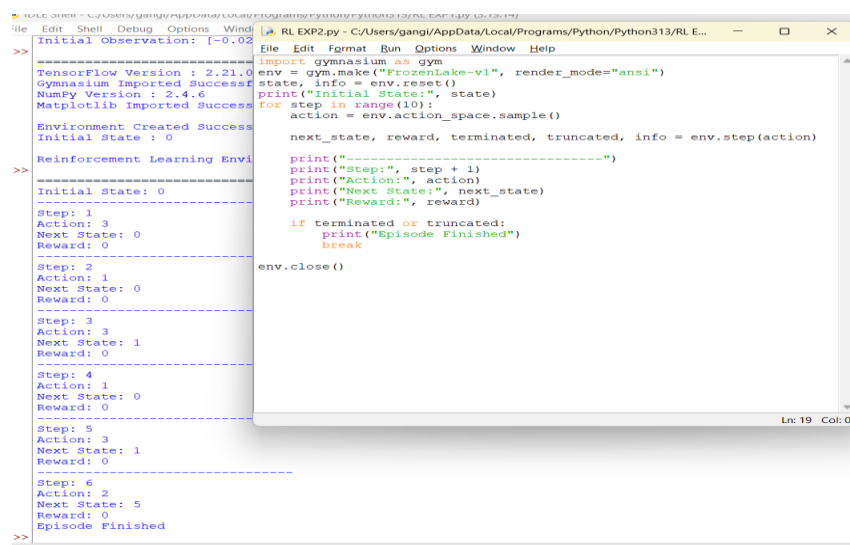
    print("Reward:", reward)

    if terminated or truncated:

        print("Episode Finished")

        break

env.close()
```



```
Initial Observation: [-0.02 0.04 0.01 0.01]
TensorFlow Version : 2.21.0
Gymnasium Imported Successfully
Numpy Version : 2.4.6
Matplotlib Imported Successfully
Environment Created Successfully
Initial State : 0
Reinforcement Learning Environment
-----
Initial State: 0
-----
Step: 1
Action: 3
Next State: 0
Reward: 0
-----
Step: 2
Action: 1
Next State: 0
Reward: 0
-----
Step: 3
Action: 3
Next State: 1
Reward: 0
-----
Step: 4
Action: 1
Next State: 0
Reward: 0
-----
Step: 5
Action: 3
Next State: 1
Reward: 0
-----
Step: 6
Action: 2
Next State: 5
Reward: 0
-----
Episode Finished
```

Experiment 3: Implementation of the Reinforcement Learning Framework

Aim: To implement the basic Reinforcement Learning framework by designing an agent that interacts with an environment through states, actions, rewards, and policies using Python.

PROGRAM:

```
import gymnasium as gym

env = gym.make("FrozenLake-v1")

state, info = env.reset()

total_reward = 0

for step in range(20):

    action = env.action_space.sample()

    next_state, reward, terminated, truncated, info = env.step(action)

    print(f"State:{state} Action:{action} Reward:{reward}")

    total_reward += reward

    state = next_state

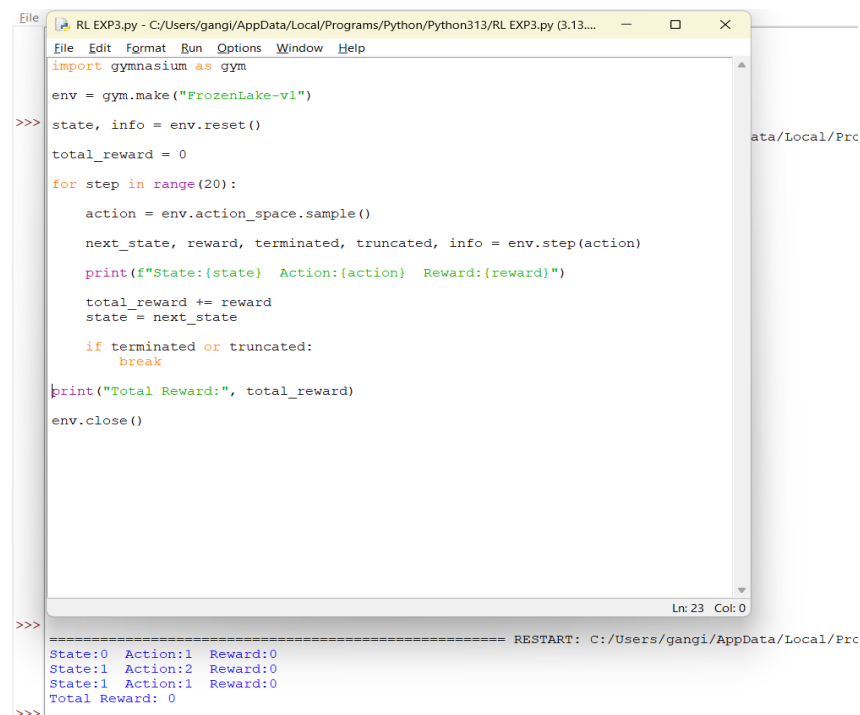
    if terminated or truncated:

        break

print("Total Reward:", total_reward)

env.close()
```

OUTPUT

A screenshot of a Python script execution window. The window title is "RL EXP3.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP3.py (3.13...)". The code is the same as in the previous block. The output shows the first few steps of the simulation: State:0 Action:1 Reward:0, State:1 Action:2 Reward:0, State:1 Action:1 Reward:0, and Total Reward: 0. The window also shows a "RESTART" message and the file path "C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP3.py (3.13...)".

```
File Edit Format Run Options Window Help
import gymnasium as gym
env = gym.make("FrozenLake-v1")
>>> state, info = env.reset()
total_reward = 0
for step in range(20):
    action = env.action_space.sample()
    next_state, reward, terminated, truncated, info = env.step(action)
    print(f"State:{state} Action:{action} Reward:{reward}")
    total_reward += reward
    state = next_state
    if terminated or truncated:
        break
print("Total Reward:", total_reward)
env.close()

Ln: 23 Col: 0

>>> ===== RESTART: C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP3.py (3.13...)
State:0 Action:1 Reward:0
State:1 Action:2 Reward:0
State:1 Action:1 Reward:0
Total Reward: 0
>>>
```

Experiment 4: Simulation of a Markov Decision Process (MDP)

Aim: To develop a simple Markov Decision Process model and simulate state transitions, reward functions, and policy execution for understanding sequential decision-making problems

PROGRAM:

```
import numpy as np

states = ["A", "B", "C"]

transition = {
    "A": {"left": "B", "right": "C"},
    "B": {"left": "A", "right": "C"},
    "C": {"left": "A", "right": "B"}
}

reward = {
    "A": 1,
    "B": 2,
    "C": 5
}

state = "A"

print("Initial State:", state)

for i in range(10):
    action = np.random.choice(["left", "right"])
    next_state = transition[state][action]
    print(f"Step {i+1}")
    print("Current State:", state)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward[next_state])
    print("-----")
    state = next_state
```

The screenshot shows a Python IDE with two windows. The left window displays the output of a simulation, showing steps 1 through 9 with current states, actions, next states, and rewards. The right window shows the Python code for the simulation, which uses NumPy to define states, transitions, and rewards, and runs a loop for 10 iterations.

```
Initial State: A
Step 1
Current State: A
Action: left
Next State: B
Reward: 2
-----
Step 2
Current State: B
Action: right
Next State: C
Reward: 5
-----
Step 3
Current State: C
Action: right
Next State: B
Reward: 2
-----
Step 4
Current State: B
Action: right
Next State: C
Reward: 5
-----
Step 5
Current State: C
Action: right
Next State: B
Reward: 2
-----
Step 6
Current State: B
Action: left
Next State: A
Reward: 1
-----
Step 7
Current State: A
Action: left
Next State: B
Reward: 2
-----
Step 8
Current State: B
Action: left
Next State: A
```

```
RL EXP4.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP4.py (3.13.14)
import numpy as np
states = ["A", "B", "C"]
transition = {
    "A": {"left": "B", "right": "C"},
    "B": {"left": "A", "right": "C"},
    "C": {"left": "A", "right": "B"}
}
reward = {
    "A": 1,
    "B": 2,
    "C": 5
}
state = "A"
print("Initial State:", state)
for i in range(10):
    action = np.random.choice(["left", "right"])
    next_state = transition[state][action]
    print(f"Step {i+1}")
    print("Current State:", state)
    print("Action:", action)
    print("Next State:", next_state)
    print("Reward:", reward[next_state])
    print("-----")
    state = next_state
```

Experiment 5: Bellman Equation Demonstration

Aim: To implement Bellman Expectation and Bellman Optimality Equations for calculating state-value and action-value functions and analyze the convergence of value estimation.

PROGRAM:

```
import numpy as np
```

```
gamma = 0.9
```

```
reward = np.array([0, 1, 5])
```

```
transition = np.array([
```

```
    [0.5,0.5,0],
```

```
    [0,0.5,0.5],
```

```
    [0.5,0,0.5]
```

```
])
```

```
V = np.zeros(3)
```

```
for iteration in range(10):
```

```
    newV = reward + gamma * transition.dot(V)
```

```
    V = newV
```

```
    print("Iteration", iteration+1)
```

```
    print(V)
```

```
print("\nFinal State Values")
```

```
print(V)
```

```

=====
Iteration 1
[0. 1. 5.]
Iteration 2
[0.45 3.7 7.25]
Iteration 3
[1.8675 5.9275 8.465 ]
Iteration 4
[3.50775 7.476625 9.649625]
Iteration 5
[ 4.94296875  8.7068125 10.92081875]
Iteration 6
[ 6.14240156  9.83243406 12.13870437]
Iteration 7
[ 7.18867603 10.8870123 13.22649767]
Iteration 8
[ 8.13405975 11.85107949 14.18682817]
Iteration 9
[ 8.99331266 12.71705844 15.04439956]
Iteration 10
[ 9.76966699 13.4926561 15.8169705 ]

Final State Values
[ 9.76966699 13.4926561 15.8169705 ]

*RL EXP5.py - C:/Users/gangji/AppData/Local/Programs/Python/Python313/RL EXP5.py
File Edit Format Run Options Window Help
import numpy as np
gamma = 0.9
reward = np.array([0, 1, 5])
transition = np.array([
    [0.5,0.5,0],
    [0,0.5,0.5],
    [0.5,0,0.5]
])
V = np.zeros(3)

for iteration in range(10):

    newV = reward + gamma * transition.dot(V)

    V = newV

    print("Iteration", iteration+1)
    print(V)

print("\nFinal State Values")
print(V)
Ln: 8 Col: 2

```

Experiment 6: Exploration vs. Exploitation Strategies

Aim: To implement ϵ -Greedy, Greedy, and Random action-selection strategies and compare their performance in balancing exploration and exploitation during Reinforcement Learning.

PROGRAM:

```
) import random
```

```
epsilon = 0.2
```

```
Q = [10, 15, 8]
```

```
for episode in range(20):
```

```
    if random.random() < epsilon:
```

```
        action = random.randint(0,2)
```

```
        print("Explore -> Action", action)
```

```
    else:
```

```
        action = Q.index(max(Q))
```

```
        print("Exploit -> Action", action)
```

```

=====
Exploit -> Action 1
Exploit -> Action 1
Exploit -> Action 1
Exploit -> Action 1
Exploit -> Action 1
Explore -> Action 2
Exploit -> Action 1
Exploit -> Action 1
Explore -> Action 0
Exploit -> Action 1
Exploit -> Action 1
Explore -> Action 1
Exploit -> Action 1
Explore -> Action 1
Exploit -> Action 1
Explore -> Action 0
Exploit -> Action 1
Exploit -> Action 1

*RL EXP6.py - C:/Users/gangji/AppData/Local/Programs/Python/Python313/RL EXP6.py (3...
File Edit Format Run Options Window Help
import random
epsilon = 0.2
Q = [10, 15, 8]
for episode in range(20):
    if random.random() < epsilon:

        action = random.randint(0,2)

        print("Explore -> Action", action)

    else:

        | action = Q.index(max(Q))

        print("Exploit -> Action", action)
Ln: 13 Col: 5

```

Experiment 7: Multi-Armed Bandit Problem

Aim: To implement the Multi-Armed Bandit problem using ϵ -Greedy and Upper Confidence Bound (UCB) algorithms and evaluate cumulative rewards obtained by different action-selection methods

PROGRAM:

```
import numpy as np

arms = [1,3,5,8]

Q = np.zeros(4)

N = np.zeros(4)

epsilon = 0.1

for i in range(100):

    if np.random.rand() < epsilon:

        action = np.random.randint(4)

    else:

        action = np.argmax(Q)

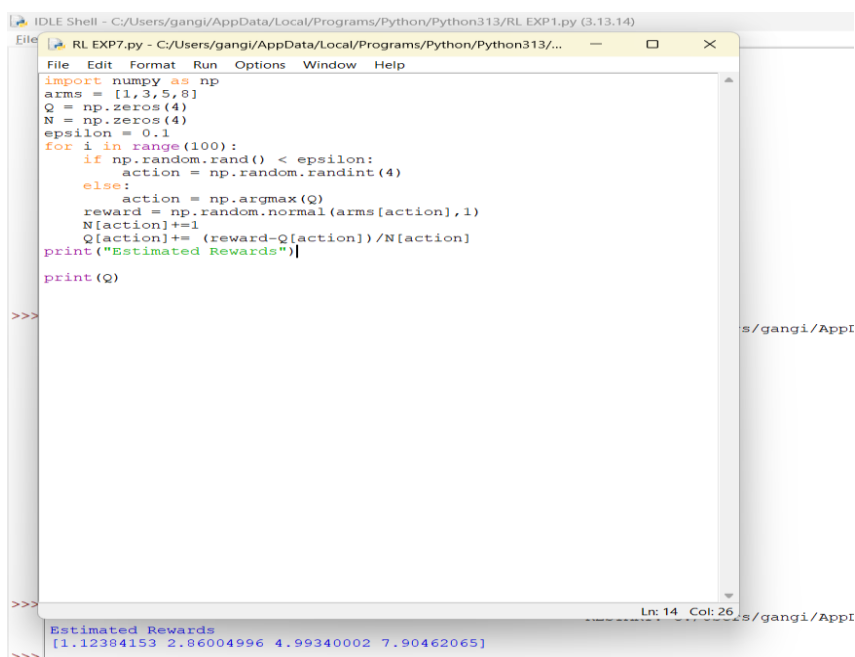
    reward = np.random.normal(arms[action],1)

    N[action]+=1

    Q[action]+= (reward-Q[action])/N[action]

print("Estimated Rewards")

print(Q)
```



```
IDLE Shell - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP1.py (3.13.14)
File Edit Format Run Options Window Help
RL EXP7.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/...
import numpy as np
arms = [1,3,5,8]
Q = np.zeros(4)
N = np.zeros(4)
epsilon = 0.1
for i in range(100):
    if np.random.rand() < epsilon:
        action = np.random.randint(4)
    else:
        action = np.argmax(Q)
    reward = np.random.normal(arms[action],1)
    N[action]+=1
    Q[action]+= (reward-Q[action])/N[action]
print("Estimated Rewards")
print(Q)
>>>
Estimated Rewards
[1.12384153 2.86004996 4.99340002 7.90462065]
>>>
```

Experiment 8: Reward Visualization in Reinforcement Learning

Aim: To simulate an RL agent in a simple environment and visualize cumulative rewards, episode rewards, and learning performance using Matplotlib graphs.

PROGRAM:

```
import matplotlib.pyplot as plt

import numpy as np

episodes = np.arange(1,51)

rewards = np.random.randint(0,100,50)

cumulative = np.cumsum(rewards)

plt.figure(figsize=(8,5))

plt.plot(episodes,cumulative)

plt.title("Cumulative Reward")

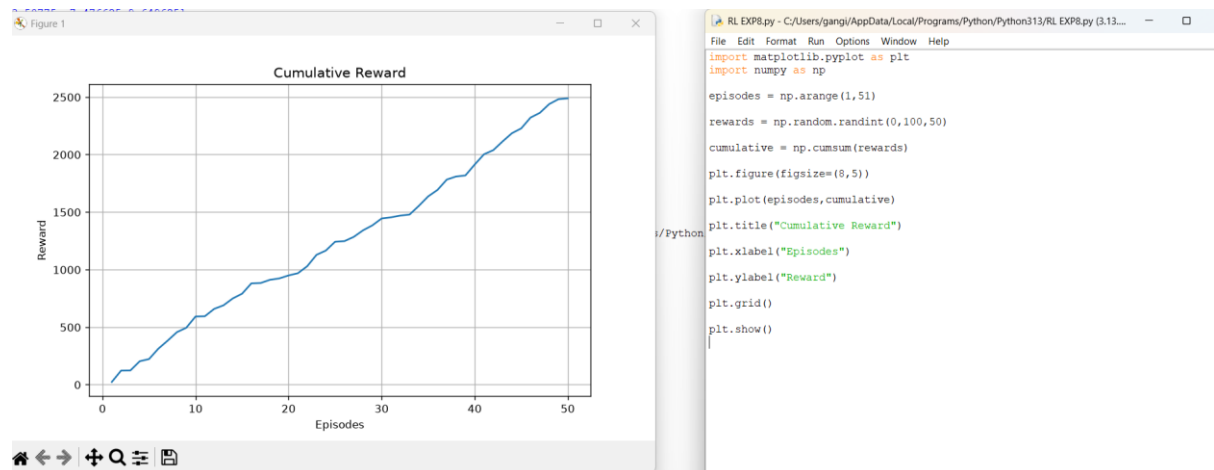
plt.xlabel("Episodes")

plt.ylabel("Reward")

plt.grid()

plt.show()
```

OUTPUT



Experiment 9: TensorFlow and Keras-Based Neural Network for RL

Aim: To build a simple feed-forward neural network using TensorFlow and Keras for approximating value functions in Reinforcement Learning environments

PROGRAM:

```
import tensorflow as tf

from tensorflow.keras.models import Sequential

from tensorflow.keras.layers import Dense

model = Sequential()

model.add(Dense(24,input_shape=(4,),activation='relu'))

model.add(Dense(24,activation='relu'))

model.add(Dense(2,activation='linear'))

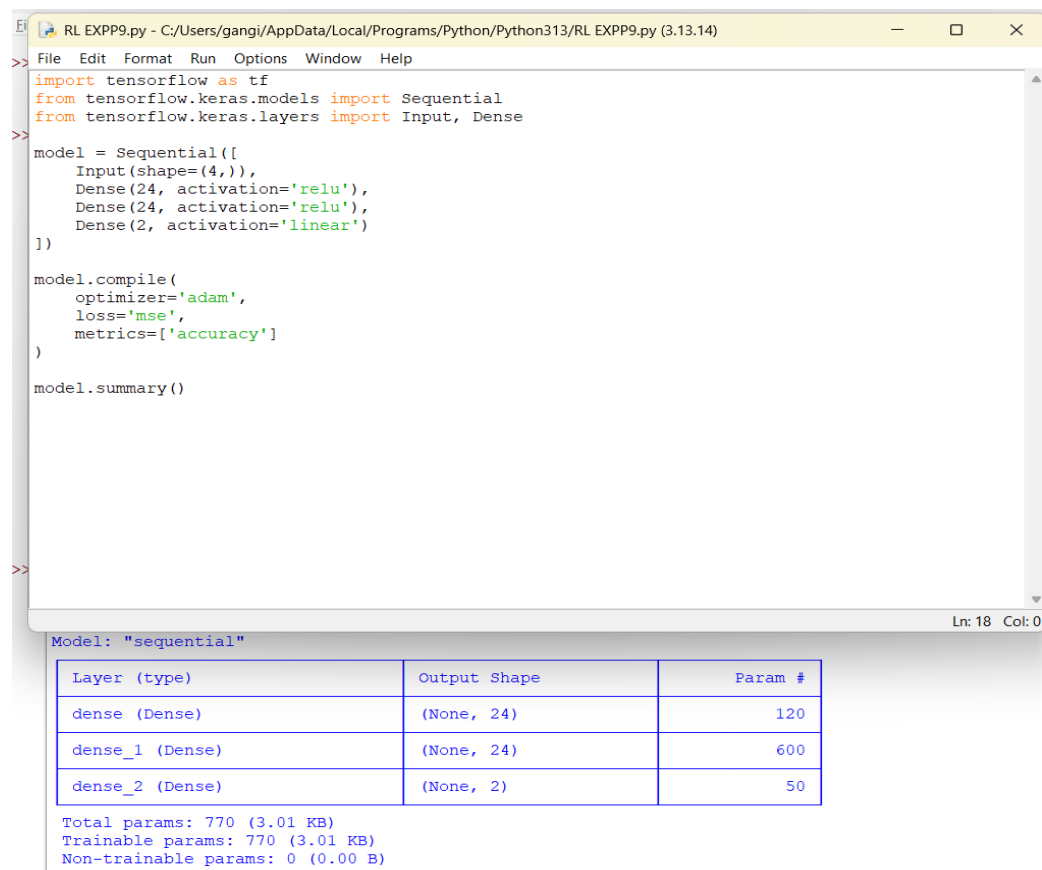
model.compile(optimizer='adam',

              loss='mse',

              metrics=['accuracy'])

model.summary()
```

OUTPUT



The screenshot shows a Python IDE window titled "RL EXPP9.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXPP9.py (3.13.14)". The code in the editor is as follows:

```
>> import tensorflow as tf
>> from tensorflow.keras.models import Sequential
>> from tensorflow.keras.layers import Input, Dense
>>
>> model = Sequential([
>>     Input(shape=(4,)),
>>     Dense(24, activation='relu'),
>>     Dense(24, activation='relu'),
>>     Dense(2, activation='linear')
>> ])
>>
>> model.compile(
>>     optimizer='adam',
>>     loss='mse',
>>     metrics=['accuracy']
>> )
>>
>> model.summary()
```

The output at the bottom of the window is:

```
Model: "sequential"
Layer (type) Output Shape Param #
dense (Dense) (None, 24) 120
dense_1 (Dense) (None, 24) 600
dense_2 (Dense) (None, 2) 50
Total params: 770 (3.01 KB)
Trainable params: 770 (3.01 KB)
Non-trainable params: 0 (0.00 B)
```

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 24)	120
dense_1 (Dense)	(None, 24)	600
dense_2 (Dense)	(None, 2)	50

Total params: 770 (3.01 KB)
Trainable params: 770 (3.01 KB)
Non-trainable params: 0 (0.00 B)

Experiment 10: Mini Reinforcement Learning Project Using CartPole

Aim: To develop a basic Reinforcement Learning agent using TensorFlow and Keras to solve the CartPole environment and evaluate its learning performance through episode rewards and success rate.

PROGRAM:

```
import gymnasium as gym

import numpy as np

env = gym.make("CartPole-v1")

episodes = 10

for episode in range(episodes):

    state, info = env.reset()

    total_reward = 0

    done = False

    while not done:

        action = env.action_space.sample()

        state, reward, terminated, truncated, info = env.step(action)

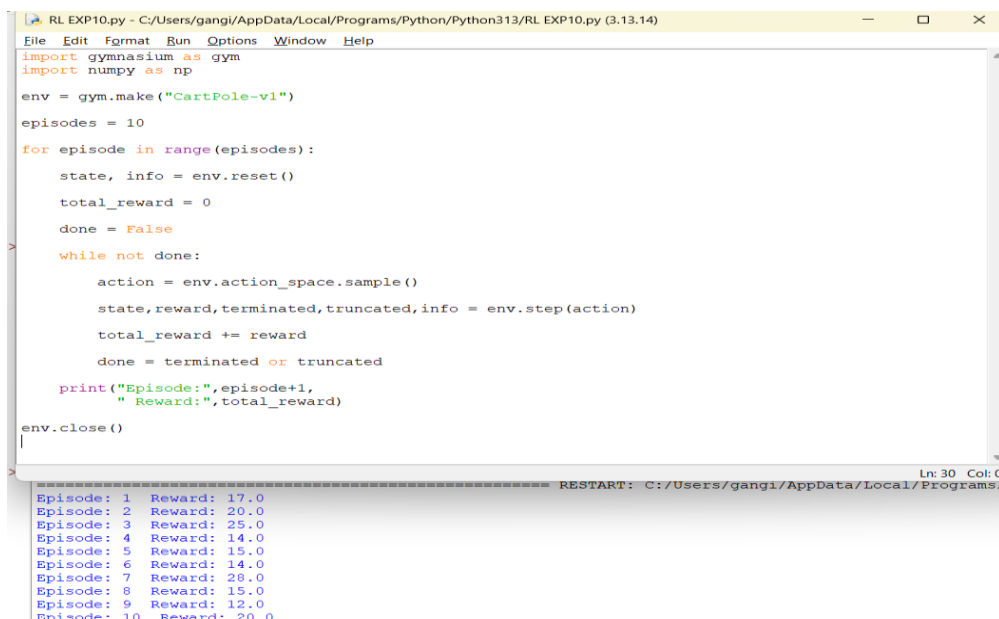
        total_reward += reward

        done = terminated or truncated

    print("Episode:", episode+1,

          " Reward:", total_reward)

env.close()
```



The screenshot shows a Jupyter Notebook window titled "RL EXP10.py - C:/Users/gangi/AppData/Local/Programs/Python/Python313/RL EXP10.py (3.13.14)". The code in the notebook is identical to the program shown above. The output at the bottom of the notebook shows the results of 10 episodes:

```
Episode: 1 Reward: 17.0
Episode: 2 Reward: 20.0
Episode: 3 Reward: 25.0
Episode: 4 Reward: 14.0
Episode: 5 Reward: 15.0
Episode: 6 Reward: 14.0
Episode: 7 Reward: 28.0
Episode: 8 Reward: 15.0
Episode: 9 Reward: 12.0
Episode: 10 Reward: 20.0
```